

K-Means en datos reales: cuando el clustering ayuda, pero no alcanza

En el cuaderno anterior trabajamos con datos sintéticos. Allí K-Means funcionó muy bien porque los grupos eran compactos, estaban separados y tenían una forma adecuada para el algoritmo.

En este cuaderno vamos a aplicar K-Means a un dataset real: **Palmer Penguins**.

La pregunta que nos va a guiar será distinta:

¿Qué ocurre cuando aplicamos K-Means a datos reales que no necesariamente cumplen las condiciones ideales del algoritmo?

Trabajaremos con variables físicas de los pingüinos, pero no usaremos la especie (`species`) para entrenar el modelo. La especie quedará reservada solamente para interpretar los clusters encontrados.

El objetivo no será demostrar que K-Means siempre funciona, sino analizar críticamente cuándo ayuda, cuándo no alcanza y por qué sus resultados deben interpretarse con cuidado.

1. Recordatorio: qué vimos en el cuaderno anterior

En el cuaderno anterior vimos que K-Means es un algoritmo de clustering. Su objetivo es formar grupos de puntos similares entre sí, sin utilizar etiquetas conocidas durante el entrenamiento.

También vimos que K-Means funciona especialmente bien cuando los grupos son compactos, tienen una forma aproximadamente circular, están bien separados y las variables están correctamente escaladas.

En este nuevo ejemplo vamos a trabajar con datos reales. Por eso, es posible que el resultado no sea tan perfecto como en el caso sintético. Esa diferencia es justamente lo que queremos analizar.

2. Importamos las librerías necesarias

Usaremos `pandas` para trabajar con tablas, `matplotlib` y `seaborn` para construir gráficos, y algunas herramientas de `scikit-learn` para escalar los datos, aplicar K-Means y calcular métricas básicas.

En este cuaderno no usaremos `Pipeline`. El objetivo es mantener cada paso visible y fácil de seguir.

```
# -----  
# Librerías para manejo de datos  
# -----  
import pandas as pd  
import numpy as np
```

```

# -----
# Librerías para visualización
# -----
import matplotlib.pyplot as plt
import seaborn as sns

# -----
# Herramientas de Scikit-learn
# -----
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# -----
# Configuración general
# -----
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

sns.set_theme(style="whitegrid")

print("Librerías importadas correctamente.")

```

3. Cargamos el dataset Palmer Penguins

El dataset **Palmer Penguins** contiene información sobre pingüinos observados en distintas islas del archipiélago Palmer, en la Antártida.

Cada fila representa un pingüino y cada columna describe alguna característica observada o medida.

Aunque el dataset incluye la columna `species`, no la usaremos para entrenar K-Means. Esto es importante: en clustering trabajamos sin etiquetas conocidas. La especie se usará recién al final para interpretar los resultados.

```

# -----
# Carga del dataset Palmer Penguins desde una URL pública
# -----
url = "https://raw.githubusercontent.com/allisonhorst/palmerpenguins/main/instantaneous_data"

df = pd.read_csv(url)

# -----
# Mostramos las primeras filas
# -----
display(df.head())

```

4. Revisamos la estructura general de los datos

Antes de seleccionar variables, revisamos la estructura del dataset.

Nos interesa conocer cuántas filas y columnas tiene, qué tipos de datos contiene y si existen valores faltantes.

Como vamos a trabajar con K-Means sin imputación ni pipelines, más adelante eliminaremos las filas incompletas en las variables seleccionadas.

```
# -----  
# Dimensiones del dataset  
# -----  
print("Cantidad de filas y columnas:")  
print(df.shape)  
  
print("\nInformación general del dataset:")  
df.info()  
  
print("\nValores faltantes por columna:")  
display(df.isna().sum())
```

5. Seleccionamos las variables para el clustering

Para aplicar K-Means vamos a utilizar solamente variables numéricas relacionadas con características físicas de los pingüinos:

- `bill_length_mm` : largo del pico.
- `bill_depth_mm` : profundidad del pico.
- `flipper_length_mm` : largo de la aleta.
- `body_mass_g` : masa corporal.

No usaremos `species` para entrenar el modelo. La incluiremos en el DataFrame preparado solo para analizar después si los clusters encontrados se parecen o no a las especies reales.

También eliminaremos las filas que tengan valores faltantes en estas columnas. En este dataset son muy pocos casos, por lo que la pérdida de información será mínima.

```
# -----  
# Variables que usaremos para K-Means  
# -----  
variables_clustering = [  
    "bill_length_mm",  
    "bill_depth_mm",  
    "flipper_length_mm",  
    "body_mass_g"  
]  
  
# -----  
# Creamos un DataFrame preparado para el análisis  
# -----  
df_clustering = df[variables_clustering + ["species"]].dropna()  
  
# Variables que verá K-Means  
X = df_clustering[variables_clustering]  
  
print("Dimensiones del dataset original:")  
print(df.shape)  
  
print("\nDimensiones del dataset preparado:")  
print(df_clustering.shape)
```

```
print("\nPrimeras filas del dataset preparado:")
display(df_clustering.head())

print("\nValores faltantes en las variables seleccionadas:")
display(X.isna().sum())
```

6. Observamos las escalas de las variables

Como K-Means se basa en distancias, la escala de las variables es muy importante.

En este caso, algunas variables están medidas en milímetros, mientras que la masa corporal está medida en gramos. Si usáramos los datos directamente, `body_mass_g` podría dominar el cálculo de distancias simplemente por tener valores mucho más grandes.

Por eso, antes de entrenar el modelo, vamos a revisar las estadísticas descriptivas y luego escalaremos los datos.

```
# -----
# Estadísticas descriptivas de las variables originales
# -----
resumen_variables = X.describe().T

display(resumen_variables)

# -----
# Gráfico de caja para comparar escalas
# -----
plt.figure(figsize=(10, 5))

sns.boxplot(data=X)

plt.title("Distribución de las variables originales")
plt.xlabel("Variable")
plt.ylabel("Valor")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

7. Estandarizamos las variables

Como vimos en el cuaderno anterior, K-Means calcula distancias entre puntos. Por eso, es importante que las variables estén en una escala comparable.

Usaremos `StandardScaler`, que transforma cada variable para que tenga media cercana a 0 y desvío estándar cercano a 1.

A partir de este punto, el entrenamiento de K-Means se realizará con los datos escalados.

```
# -----
# Creamos y aplicamos el escalador
# -----
escalador = StandardScaler()

X_escalado = escalador.fit_transform(X)
```

```

# -----
# Convertimos el resultado a DataFrame
# -----
X_escalado_df = pd.DataFrame(
    X_escalado,
    columns=variables_clustering,
    index=X.index
)

# -----
# Verificamos el resultado
# -----
display(X_escalado_df.head())

display(X_escalado_df.describe().T)

```

8. Entrenamos K-Means con $k = 3$

En este dataset sabemos que existen tres especies de pingüinos. Sin embargo, K-Means no usará esa información.

Vamos a entrenar un modelo con $k = 3$ para preguntarnos si, usando solamente las variables físicas, el algoritmo logra formar tres grupos parecidos a las especies reales.

Esta decisión no significa que K-Means conozca las especies. Solo le estamos pidiendo que forme tres clusters a partir de las distancias entre los puntos.

```

# -----
# Creamos y entrenamos K-Means con  $k = 3$ 
# -----
kmeans_3 = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

kmeans_3.fit(X_escalado_df)

# -----
# Obtenemos las etiquetas de cluster
# -----
clusters_k3 = kmeans_3.labels_

print("Modelo K-Means entrenado correctamente.")
print("Primeras etiquetas de cluster asignadas:")
print(clusters_k3[:20])

```

9. Agregamos los clusters al dataset

El modelo asignó a cada pingüino una etiqueta de cluster: 0, 1 o 2.

Estos números no tienen un significado propio. No representan especies ni tienen un orden especial. Son simplemente nombres internos que K-Means utiliza para identificar los grupos encontrados.

Ahora vamos a agregar esas etiquetas al DataFrame para poder analizarlas.

```
# -----  
# Creamos una copia del DataFrame preparado  
# -----  
df_k3 = df_clustering.copy()  
  
# -----  
# Agregamos el cluster asignado  
# -----  
df_k3["cluster_k3"] = clusters_k3  
  
# -----  
# Mostramos las primeras filas  
# -----  
display(df_k3.head())
```

10. Analizamos cuántos pingüinos quedaron en cada cluster

Una primera revisión consiste en contar cuántos registros quedaron asignados a cada cluster.

Esto no nos dice todavía si el clustering es bueno o malo, pero permite observar si el algoritmo formó grupos de tamaños razonables o si algún cluster quedó demasiado pequeño o demasiado grande.

```
# -----  
# Conteo de registros por cluster  
# -----  
conteo_clusters = df_k3["cluster_k3"].value_counts().sort_index()  
  
display(conteo_clusters)  
  
# -----  
# Gráfico de barras  
# -----  
plt.figure(figsize=(7, 4))  
  
sns.barplot(  
    x=conteo_clusters.index,  
    y=conteo_clusters.values  
)  
  
plt.title("Cantidad de pingüinos por cluster con k = 3")  
plt.xlabel("Cluster")  
plt.ylabel("Cantidad de registros")  
plt.tight_layout()  
plt.show()
```

11. Visualizamos los clusters encontrados

Ahora vamos a graficar los clusters usando dos variables físicas:

- `flipper_length_mm` : largo de la aleta.

- `body_mass_g` : masa corporal.

Aunque K-Means fue entrenado con cuatro variables, este gráfico nos permite observar una parte importante de la estructura de los datos.

Cada punto representa un pingüino y el color indica el cluster asignado por K-Means.

```
# -----  
# Gráfico de clusters encontrados por K-Means  
# -----  
plt.figure(figsize=(8, 5))  
  
sns.scatterplot(  
    data=df_k3,  
    x="flipper_length_mm",  
    y="body_mass_g",  
    hue="cluster_k3",  
    palette="viridis",  
    s=80  
)  
  
plt.title("Clusters encontrados por K-Means con k = 3")  
plt.xlabel("Largo de la aleta (mm)")  
plt.ylabel("Masa corporal (g)")  
plt.legend(title="Cluster")  
plt.tight_layout()  
plt.show()
```

12. Comparamos con las especies reales

K-Means no usó `species` para entrenar. Sin embargo, como en este dataset sí conocemos las especies, podemos usarlas después para interpretar el resultado.

Esto no convierte el problema en supervisado. La especie funciona aquí como una referencia externa que nos ayuda a responder una pregunta:

¿Los clusters encontrados por K-Means se parecen a las especies reales?

Vamos a graficar los mismos puntos, pero ahora coloreados por especie.

```
# -----  
# Gráfico coloreado por especie real  
# -----  
plt.figure(figsize=(8, 5))  
  
sns.scatterplot(  
    data=df_k3,  
    x="flipper_length_mm",  
    y="body_mass_g",  
    hue="species",  
    style="species",  
    s=80  
)  
  
plt.title("Especies reales según largo de aleta y masa corporal")  
plt.xlabel("Largo de la aleta (mm)")  
plt.ylabel("Masa corporal (g)")
```

```
plt.legend(title="Especie")
plt.tight_layout()
plt.show()
```

13. ¿Los clusters coinciden con las especies reales?

Los gráficos sugieren que K-Means separa bastante bien a un grupo de pingüinos con mayor largo de aleta y mayor masa corporal. Ese grupo parece relacionarse con la especie `Gentoo`.

Sin embargo, también se observa que `Adelie` y `Chinstrap` aparecen más mezclados entre sí.

Para analizarlo con mayor precisión, construiremos una tabla cruzada entre `cluster_k3` y `species`, y luego la representaremos con un mapa de calor.

```
# -----
# Tabla cruzada entre clusters y especies reales
# -----
tabla_cluster_especie = pd.crosstab(
    df_k3["cluster_k3"],
    df_k3["species"]
)

display(tabla_cluster_especie)

# -----
# Mapa de calor
# -----
plt.figure(figsize=(8, 5))

sns.heatmap(
    tabla_cluster_especie,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Relación entre clusters de K-Means y especies reales")
plt.xlabel("Especie real")
plt.ylabel("Cluster asignado por K-Means")
plt.tight_layout()
plt.show()
```

14. Interpretamos la comparación entre clusters y especies

La tabla cruzada permite ver con claridad qué encontró K-Means.

Uno de los clusters suele coincidir muy bien con la especie `Gentoo`. Esto tiene sentido si recordamos los gráficos: los pingüinos `Gentoo` se distinguen bastante por su mayor largo de aleta y mayor masa corporal.

Sin embargo, los otros clusters no coinciden perfectamente con **Adelie** y **Chinstrap** . Estas dos especies presentan mayor solapamiento en las variables físicas seleccionadas.

Este resultado muestra una idea fundamental:

K-Means no intenta encontrar especies. K-Means agrupa puntos según su cercanía en el espacio de variables que le damos.

Por eso, un dataset puede funcionar muy bien para clasificación supervisada y, al mismo tiempo, no ser ideal para clustering con K-Means.

15. Una métrica interna: silhouette score

Aunque en este caso podemos comparar con las especies reales, en un verdadero problema de clustering normalmente no tendríamos etiquetas externas.

Por eso, también podemos calcular una métrica interna como el **silhouette score**.

Esta métrica mide si los puntos están más cerca de los elementos de su propio cluster que de los elementos de otros clusters. Sus valores van de -1 a 1:

- valores cercanos a 1 indican clusters bien definidos;
- valores cercanos a 0 indican puntos en zonas de frontera;
- valores negativos sugieren posibles asignaciones problemáticas.

Esta métrica ayuda, pero no reemplaza la interpretación visual ni el análisis del problema.

```
# -----  
# Cálculo del silhouette score para k = 3  
# -----  
silueta_k3 = silhouette_score(X_escalado_df, clusters_k3)  
  
print(f"Silhouette score para k = 3: {silueta_k3:.4f}")
```

16. Cierre: cuando K-Means ayuda, pero no alcanza

En este cuaderno aplicamos K-Means a un dataset real.

A diferencia del cuaderno anterior, donde los datos sintéticos estaban generados en condiciones ideales, aquí trabajamos con mediciones reales de pingüinos.

El resultado fue muy interesante:

- K-Means logró identificar con mucha claridad un grupo que se relaciona con **Gentoo** .
- Sin embargo, no logró separar perfectamente **Adelie** y **Chinstrap** .
- Los clusters encontrados no coinciden exactamente con las especies reales.

Esto no significa que el algoritmo esté mal aplicado. Significa que los datos no forman tres grupos perfectamente compactos y separados según las variables que elegimos.

La conclusión principal es que el clustering debe interpretarse con cuidado.

K-Means puede encontrar patrones útiles, pero no descubre automáticamente “la verdad” del dataset. Sus resultados dependen de las variables utilizadas, de la escala, de la forma de los grupos y del valor de k elegido.

En este caso, el algoritmo ayudó a encontrar una parte de la estructura, pero no alcanzó para reproducir completamente las especies reales